

Rapport de projet

Conteneurisation et déploiement d'un portfolio sur AWS

Docker | Terraform | AWS ECS Fargate | GitHub Actions

Contexte

Ce projet a pour objectif de mettre en place une infrastructure DevOps complète autour d'un portfolio personnel développé avec React, Vite, TypeScript et Tailwind CSS. L'idée est de passer d'un simple site hébergé sur Netlify à une infrastructure cloud maîtrisée de bout en bout, en utilisant Docker, AWS et Terraform.

L'objectif n'est pas uniquement de déployer une application, mais de démontrer la capacité à mettre en place un pipeline DevOps complet : conteneurisation, infrastructure as code, déploiement automatisé et observabilité.

1. Conteneurisation avec Docker

Principe du build multi-stage

Le Dockerfile utilise un build multi-stage : une première image Node.js compile le projet Vite, et une deuxième image nginx-unprivileged sert uniquement les fichiers statiques générés. Cette approche réduit la taille de l'image finale de plus d'1 Go à seulement 77 Mo, en excluant tous les outils de build de l'image de production.

L'image nginx-unprivileged est utilisée pour exécuter le serveur en tant qu'utilisateur non-root (uid 101), ce qui réduit la surface d'attaque en production.

```
anas@debian:~/dev/portefolio-devops/infra$ sudo docker images | grep portfolio
[sudo] Mot de passe de anas :
portfolio:dev    aac46a2b2cf7    77.6MB    22.8MB    U
anas@debian:~/dev/portefolio-devops/infra$
```

L'image finale pèse 77.6 Mo grâce au build multi-stage.

Healthcheck et le piège IPv4/IPv6

Un point important a été rencontré pendant cette étape : le healthcheck du conteneur échouait systématiquement malgré un nginx qui fonctionnait correctement. La cause était un problème de résolution DNS à l'intérieur du conteneur — localhost se résolvait en IPv6 (::1) alors que nginx n'écoutait que sur IPv4.

La solution a été de remplacer localhost par 127.0.0.1 dans la commande de sonde du HEALTHCHECK. Ce type de détail, invisible en apparence, est exactement ce qui différencie un environnement de développement d'un environnement de production.

```
anas@debian:~/dev/portefolio-devops/infra$ sudo docker run -d -p 8080:8080 portfolio:dev
30dc3e554025d11fd3867e783b225f2d1d749c428edcb11c0e61c1503f8f8936
anas@debian:~/dev/portefolio-devops/infra$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
30dc3e554025   portfolio:dev  "/docker-entrypoint..."  7 seconds ago  Up 7 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  admiring_galois
anas@debian:~/dev/portefolio-devops/infra$
```

Le conteneur affiche le statut (healthy) après correction du healthcheck.

2. Infrastructure as Code avec Terraform

Backend distant : S3 + DynamoDB

Avant le premier terraform init, un bucket S3 et une table DynamoDB sont créés manuellement pour stocker l'état Terraform à distance. Ce mécanisme est essentiel pour deux raisons :

- Le state S3 est versionné et chiffré, ce qui permet de revenir en arrière en cas d'erreur.
- La table DynamoDB sert de verrou : elle empêche deux terraform apply de tourner simultanément et de corrompre l'état, ce qui est indispensable en environnement d'équipe.

Ressources provisionnées (19 au total)

- VPC avec deux sous-réseaux publics sur deux zones de disponibilité (eu-west-3a et eu-west-3b)
- Internet Gateway et table de routage pour l'accès public
- Deux Security Groups : un pour l'ALB (port 80), un pour ECS (port 8080 depuis l'ALB uniquement)
- Dépôt ECR pour stocker les images Docker
- Cluster ECS Fargate avec task definition et service
- Application Load Balancer avec target group et listener HTTP
- Log group CloudWatch pour les logs du conteneur (rétention 7 jours)
- Rôle IAM pour l'exécution des tâches ECS

```
Plan: 19 to add, 0 to change, 0 to destroy.
```

Le plan Terraform liste 19 ressources à créer, sans aucune modification ni suppression.

```
Apply complete! Resources: 19 added, 0 changed, 0 destroyed.
```

Après confirmation, Terraform crée l'ensemble des ressources.

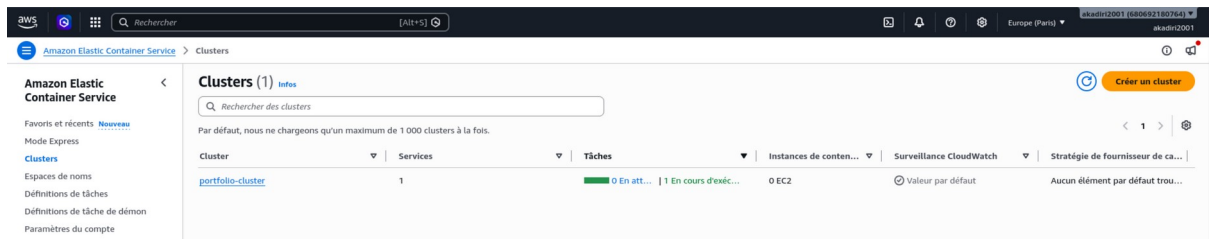
3. Push de l'image et déploiement sur ECS

Une fois l'infrastructure créée, l'image Docker est taguée et poussée sur le registre ECR. ECS Fargate récupère ensuite cette image pour démarrer le conteneur en production. Le service ECS est configuré avec un desired count de 1 tâche, répartie sur deux sous-réseaux pour la haute disponibilité.

L'Application Load Balancer distribue le trafic HTTP entrant vers la tâche Fargate via un target group. Un health check est configuré sur la route / pour s'assurer que le conteneur répond correctement avant de lui envoyer du trafic.

```
anas@debian:~/dev/portefolio-devops/infra$ sudo docker push 680692180764.dkr.ecr.eu-west-3.amazonaws.com/portfolio:latest
The push refers to repository [680692180764.dkr.ecr.eu-west-3.amazonaws.com/portfolio]
945f5aaec3d7: Pushed
513bae533eb2: Pushed
d8d5479b8af9: Pushed
d2ef8da8d838: Pushed
048c29a61317: Pushed
576fe098e846: Pushed
8f7ead5f2dc0: Pushed
400c1de759f4: Pushed
b16687ec2b7b: Pushed
0a0ff06810f1: Pushed
f79056b7935e: Pushed
f18232174bc9: Pushed
latest: digest: sha256:aac46a2b2cf71247f3ab83b7566d775e567f2d72f684db11f8d616e5ecb4292c size: 856
```

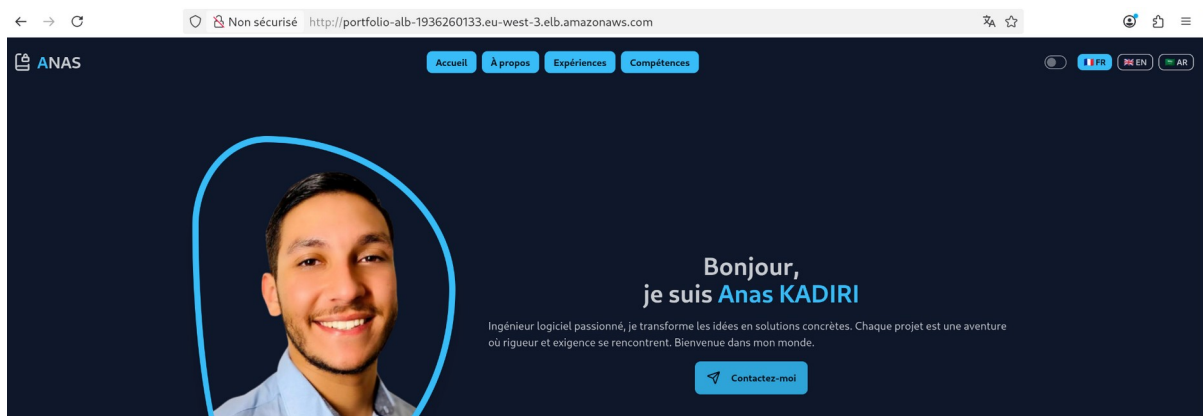
L'image portfolio:latest est bien présente dans le registre ECR.



Le service ECS affiche 1 tâche en cours d'exécution (RUNNING).



La cible enregistrée dans le target group est en statut healthy.



Le portfolio est accessible publiquement via l'URL du load balancer.

4. Pipeline CI/CD avec GitHub Actions

Un workflow GitHub Actions est mis en place pour automatiser le déploiement à chaque push sur la branche main. Les credentials AWS sont stockés dans les secrets du dépôt GitHub et ne sont jamais écrits en clair dans le code.

Étapes du pipeline

- Récupération du code source (checkout)
- Configuration des credentials AWS depuis les secrets GitHub
- Authentification au registre ECR
- Build de l'image Docker taguée avec le SHA du commit (traçabilité)
- Push de l'image sur ECR
- Forçage du redéploiement du service ECS

Gestion des coûts

Cette infrastructure n'est pas éligible au free tier AWS (ALB + Fargate engendrent des coûts). La bonne pratique appliquée ici est de détruire l'infrastructure après chaque session avec terraform destroy, et de la recréer très rapidement quand nécessaire. Cela illustre un principe fondamental du DevOps : l'infrastructure doit être éphémère et entièrement reproductible depuis le code.

Conclusion — Les grandes étapes

Ce projet démontre qu'il est possible de mettre en place une infrastructure cloud complète et reproductible autour d'une application existante, sans modifier une seule ligne du code applicatif. Voici un récapitulatif des grandes étapes réalisées :

Etape	Technologie	Ce qui a été mis en place
Conteneurisation	Docker	Build multi-stage, nginx-unprivileged, healthcheck IPv4
Registre	AWS ECR	Stockage privé des images Docker avec scan activé
Infrastructure	Terraform	VPC, ECS Fargate, ALB, IAM, CloudWatch — 19 ressources
State Terraform	S3 + DynamoDB	State distant, versionné, chiffré, avec verrouillage
Déploiement	AWS ECS Fargate	Exécution serverless du conteneur en production
Automatisation	GitHub Actions	Pipeline build > push > deploy déclenché au push
Sécurité	IAM + Security Groups	Moindre privilège, secrets hors du code source
Gestion des coûts	terraform destroy	Infrastructure éphémère, recréable en 5 minutes

Ce projet illustre une approche DevOps moderne : chaque composant est défini en code, versionné, et reproductible. L'infrastructure peut être détruite et recréée en quelques minutes, ce qui est la définition même d'une infrastructure bien conçue.